课　程　实　践　任　务

开封菊花视觉分类
数据集采集与构建任务书

数据集工程 · GPS 元数据 · YOLO26 模型训练

课程名称　Python 开发与应用

关联章节　第 8 章 数据分析　第 9 章 人工智能基础　第 10 章 应用案例

适用对象　25 级软件工程（工程智能软件方向）

任务类型　小组实践 · 数据集构建与模型训练

建议周期　4—5 周

河南大学 建筑工程学院 · 智能建造与智慧交通教研室

二〇二六年十一月

目　录

一、任务背景与目标..... 1

二、任务总体要求..... 2

三、小组组建与差异化要求..... 3

3.1 分组方式..... 3

3.2 差异化要求一：数据集相似度低于 70%..... 4

3.3 差异化要求二：模型不得相同..... 5

3.4 为什么要求"不同"..... 6

四、菊花类别的确定与命名..... 6

4.1 如何选定 10 个类别..... 6

4.2 类别文件夹命名规则..... 7

4.3 单张图片文件命名..... 8

五、拍摄规范..... 8

5.1 设备与相机设置..... 8

5.2 经纬度必须保留（本任务最容易被破坏的环节）..... 9

5.3 构图与拍摄角度..... 10

5.4 拍摄数量分配建议..... 11

5.5 禁止事项..... 11

六、数据集组织结构..... 12

6.1 压缩包顶层结构..... 12

6.2 为什么采用这个结构..... 13

6.3 完整目录树示例（含具体文件名）..... 13

七、训练集、验证集、测试集的划分..... 14

7.1 划分比例与数量核算..... 14

7.2 防泄漏原则（本任务最关键的规则）..... 15

7.3 划分后必须自查 16
八、元数据表 labels.csv 16
8.1 字段定义 16
8.2 样例 17
8.3 生成方式 17
九、执行步骤 17
阶段 0：准备（1 天） 18
阶段 1：采点与拍摄（2—3 天） 18
阶段 2：原图归集与清洗（1 天） 19
阶段 3：命名与归类（1 天） 19
阶段 4：划分复核（半天） 19
阶段 5：生成元数据与说明文档（半天） 20
阶段 6：环境配置与训练（2—3 天） 20
阶段 7：打包与自检（1 天） 20
十、环境配置与依赖安装 20
10.1 硬件与系统要求 21
10.2 第一步：创建独立的虚拟环境 21
10.3 第二步：安装 PyTorch 22
10.4 第三步：安装 Ultralytics 23
10.5 第四步：设置环境变量 24
10.6 第五步：目录准备 25
10.7 环境配置常见问题 26
十一、YOLO26 分类模型训练 26
11.1 先理解一件事：分类任务的数据集不需要 YAML 26
11.2 类别索引按文件夹名字母序生成 27

11.3 训练脚本（完整版）..... 28
11.4 训练参数的取舍..... 30
11.5 读懂训练输出..... 31
11.6 训练后评估：三个集合分别测..... 32
11.7 单张图片预测..... 34
11.8 断点续训..... 35
十二、结果分析与可视化..... 35
12.1 必须产出的四张图..... 35
12.2 怎么读混淆矩阵..... 36
12.3 错误样本分析..... 36
12.4 建议做的对比实验（加分）..... 37
十三、提交前自检清单..... 37
十四、常见错误与避坑指南..... 39
14.1 数据集相关..... 39
14.2 环境与训练相关..... 40
十五、提交要求..... 41
15.1 压缩包命名..... 41
15.2 压缩包内容..... 41
15.3 README.md 内容要求（六部分）..... 42
15.4 提交方式与截止..... 43
十六、评分标准..... 43
加分项（最高 +8 分）..... 44
扣分项..... 44
附录 A：环境配置速查与排错..... 45
A.1 一条命令走完全流程（有 NVIDIA 显卡）..... 45

A.2 环境自检脚本 45
A.3 报错对照表 46
附录 B：经纬度提取与数据集检查脚本 47
B.1 用 Windows 属性快速查看（无需编程） 47
B.2 批量提取 EXIF 并生成 labels.csv 47
B.3 数据集结构自查 49
B.4 泄漏自查 50
附录 C：数据集相似度核查脚本 51
C.1 原理 51
C.2 核查脚本 51
C.3 判据补充：总相似度不够，还要看"完全重复"比例 53
C.4 使用说明 54
C.5 检测能力与局限 54
附录 D：数据规模参考 55
附录 E：名词解释 56

任务要点速览

微信里不行

10 类	3 人	100%	YOLO26
菊花类别数	每组人数	GPS 完整率	训练框架
形态差异可辨	组间须差异化	每张必须有经纬度	分类模型 -cls

本任务的核心产出是一个结构规范、可直接被训练框架读取的 ZIP 数据集包，外加一套可复现的训练代码与结果分析。评分不以模型精度为依据，而以数据集的规范性、训练过程的完整性与是否存在数据泄漏为依据。三项硬性否决条件：使用网络图片或 AI 生成图片、伪造 GPS 坐标、结构无法被框架加载。

一、任务背景与目标

本任务面向《Python 开发与应用》课程第 9—10 章的机器学习与深度学习内容，要求学生**走出教室、走进开封的菊花展区，亲手采集一份可用于图像分类训练的真实数据集，并用 YOLO26 完成训练与评估**。

开封是中国菊花名城，自 1983 年起举办菊花文化节，龙亭公园、清明上河园、铁塔公园均为主要展区。每年十月至十一月，展区内菊花品种逾千，形态、花色、瓣型差异显著，是构建细粒度图像分类数据集的理想对象。

任务的最终产出不是一个单独的文件，而是**一个结构规范、可直接被训练框架读取的数据集压缩包，加上一套可复现的训练代码与训练记录**。学生需要完成从拍摄、筛选、归类、命名、划分、打包，到配置环境、训练模型、分析结果的全过程，并保证每张照片携带真实的 GPS 经纬度信息。

本任务要达到四个目标：

1. **数据能力**：掌握图像数据集的采集规范、命名规范与目录组织规范，理解"数据集质量决定模型上限"这一基本事实。

2. **工程能力**：能够按工业界通用的 `train/val/test` 目录结构组织数据，并用脚本完成批量重命名、EXIF 提取与集合划分。
3. **建模能力**：能够独立完成深度学习环境配置，用 YOLO26 分类模型完成训练、验证与测试，读懂训练曲线与混淆矩阵。
4. **科学素养**：理解并主动规避**数据泄漏**——这是图像分类任务中最容易犯、后果最严重的错误，详见第七章。

> 需要明确的一点：本任务不追求模型精度，追求的是**过程的规范性**。一份 300 张但结构严谨、无泄漏、元数据完整、训练可复现的数据集与代码，评分高于一份 1000 张但混杂无序、训练结果无法重现的提交。

二、任务总体要求

项目	要求	说明
分组方式	**3 人一组**	见第三章，组内分工须写入 README
菊花类别数	**不少于 10 类**	类别需形态差异可辨，不能是同一品种的不同颜色
每类照片数	**不少于 30 张**，建议 40—60 张	低于 30 张时模型难以学到稳定特征
数据集总规模	**不少于 300 张**，建议 400—600 张	含训练、验证、测试三部分
拍摄设备	**手机后置主摄**，禁用网络图片	必须为本组现场拍摄，保留原始 EXIF
GPS 经纬度	**每张必须有**，完整率要求 100%	见第 5.2 节，这是硬性要求
图像格式	JPG（JPEG）	若手机输出 HEIC，需转换并保留 EXIF
图像尺寸	长边 1024—2048 像素	过小损失细节，过大影响加载速度
单张体积	300 KB—2 MB	全包体积控制在 500 MB 以内
目录结构	压缩包内 `train/`、`val/`、`test/` 三个子集	每个子集下按类别分子文件夹

项目	要求	说明
划分比例	训练 ： 验证 ： 测试 = 8 ： 1 ： 1	每类测试集不少于 5 张
元数据表	必须提交 `labels.csv`	含文件名、类别、经纬度、拍摄时间
训练框架	**Ultralytics YOLO26 分类模型**	型号 `yolo26n-cls` / `yolo26s-cls` / `yolo26m-cls` 任选，见第十一章
训练代码	**必须提交，且能一键复现**	含环境说明、随机种子、完整超参
差异化要求	**组间数据集相似度 < 70%，模型不得相同**	见第三章与附录 C
提交形式	**单个 ZIP 压缩包**	命名见第十五章

三、小组组建与差异化要求

3.1 分组方式

本任务按 **3 人一组** 完成。3 人是经过权衡的下限，理由如下：

- **拍摄量**：12 类 × 45 张 ≈ 540 张，含往返展区、构图调整、反复补拍，单人需 15 小时以上；3 人分担到每人约 5 小时，可在 3—4 周内完成。
- **标注一致性**：菊花形态辨识需要交叉复核。同一株花由两人独立判断类别，不一致时第三人裁定，能显著降低标错率。
- **工程分工**：数据采集、代码与训练、文档与汇报三块可以并行。

组内分工必须写入 README，且分工要落到具体动作，不能写"甲负责采集、乙负责代码"这种模糊表述。合格的分工示例：

成员	学号	主要职责	具体交付物
甲	2025001	类别方案与外业采集	类别清单、GPS 抽检记录、`_raw/` 原始库

成员	学号	主要职责	具体交付物
乙	2025002	数据处理与训练	`labels.csv`、划分脚本、训练脚本与日志
丙	2025003	结果分析与文档	混淆矩阵分析、错误样本集、README 与汇报 PPT

3.2 差异化要求一：数据集相似度低于 70%

同一班级内，任意两个小组的数据集**相似度必须低于 70%**。

这条要求的实质是防止"复制别人的照片改个名"。判断方法不是看文件名，而是看**图像内容本身**——用感知哈希（perceptual hash）比对，即使图片被压缩、缩放、轻微裁剪或改色，也能识别为同一张。

相似度的定义：

对 A、B 两个数据集，取 A 中每一张图，在 B 中寻找最相似的一张。若两者的哈希汉明距离不超过阈值（默认 8），则判定这张图为"重复"。相似度按下面公式计算：

代码

```
相似度 = A 与 B 重复的图片数 / min(A 的总张数, B 的总张数) × 100%
```

判定标准：

相似度	判定	处理
< 30%	通过	属于正常的"同一展区同一批花"重合，可接受
30%—70%	警告	需说明重合原因，可能扣分
≥ 70%	不通过	判定为数据集雷同，该项按 0 分计

> 阈值选 8 是工程经验值。dHash 为 64 位，8 位差异相当于允许轻微压缩与缩放，但不会把两朵不同的花误判为同一张。核查脚本见附录 C。

注意：不同小组在同一展区拍摄，拍到同几株花是正常的。真正的红线是"未

经拍摄直接使用他人照片"。因此**建议各组自选不同的展区或不同的时间段**，从源头减少重合。

3.3 差异化要求二：模型不得相同

各组提交的模型必须互不相同。判定依据是**模型指纹**，由四项组成：

维度	可选项
网络规模	`yolo26n-cls`（2.8M 参数）/ `yolo26s-cls`（6.7M）/ `yolo26m-cls`（11.6M）
输入尺寸	224 / 320 / 448
微调策略	全量微调 / 冻结主干前 N 层（`freeze`）
增强策略	默认 randaugment / 关闭自动增强改用 HSV / 使用 letterbox 保持长宽比

四项中至少三项不同，才视为不同模型。举例：

- 甲组：`yolo26n-cls` + 224 + 全量 + randaugment
- 乙组：`yolo26s-cls` + 320 + 冻结前 4 层 + 关闭自动增强
- 丙组：`yolo26m-cls` + 448 + 全量 + letterbox

以上三组互相满足"三项不同"。

组间须填写下表并在提交时一并附上（写入 README 第五章）：

项目	本组取值
模型型号	
输入尺寸	
微调策略（freeze 层数）	
增强策略	
优化器与学习率	
训练轮数	

项目	本组取值
模型指纹（6 项拼接哈希前 8 位）	

指纹计算方式见第十一章 11.6 节。教师可用指纹快速核查是否有两组完全相同。

3.4 为什么要求"不同"

这不是为了增加难度，而是为了让**结果对比有意义**。

如果全班都用 `yolo26n-cls` + 224 + 默认参数，最终准确率都会在 90% 附近，无法判断谁的方案更好，也无法让学生理解"网络规模、输入分辨率、微调深度各自带来什么影响"。

差异化之后，各组的结果汇总成一张表，就能看出规律：比如 `yolo26m-cls` 在 448 输入下准确率是否真的更高、冻结主干是否显著降低过拟合。**这张汇总表本身，就是一份比单组结果更有价值的实验报告。**

四、菊花类别的确定与命名

4.1 如何选定 10 个类别

类别划分依据**形态特征**，而不是"哪朵花好看"。以下四种划分维度任选其一为主，但必须**在说明文档中写明所依据的维度**：

1. **瓣型**：平瓣、匙瓣、管瓣、畸瓣（如"绿牡丹"为畸瓣、"帅旗"为平瓣）。
2. **花型**：单瓣型、托桂型、球型、莲座型、飞舞型。
3. **花色**：黄色系、白色系、粉色系、红色系、绿色系、复色系。
4. **栽培类型**：大花秋菊、多头菊、案头菊、悬崖菊、造型菊。

推荐做法：以"花色 + 瓣型"组合命名，例如"黄色平瓣""白色管瓣""绿色畸瓣"，这样类别之间差异明显，模型容易学，也符合展区实际分布。

> 常见误区：把同一品种的"初开"和"盛开"当作两个类别。这是生长阶段差异，不是品种差异，会让模型学到与分类目标无关的特征。

开封展区常见传统名菊包括绿牡丹、墨荷、帅旗、黄袍加身、绿云、凤凰振羽、西湖柳月等，可作为选题参考。**具体品种名以展区标牌为准，不得凭印象编造。**

4.2 类别文件夹命名规则

类别文件夹名必须满足三个条件：**英文或拼音、全小写、无空格**。

格式：`{两位序号}_{类别名拼音}`

序号	类别（示例）	合法文件夹名	不合法写法
01	黄色平瓣	`01_huangse_pingban`	`01 黄色平瓣`、`1-黄色`
02	白色管瓣	`02_baise_guanban`	`02.白色管瓣`
03	绿色畸瓣	`03_lvse_jiban`	`03_绿色畸瓣`
04	粉色莲座	`04_fense_lianzuo`	`04 粉色（莲座）`
05	红色球型	`05_hongse_qiuxing`	`5_红球`
06	复色飞舞	`06_fuse_feiwu`	`06_复色/飞舞`
07	黄色管瓣	`07_huangse_guanban`	`07_黄管`
08	白色平瓣	`08_baise_pingban`	`08_白平`
09	紫色匙瓣	`09_zise_chiban`	`09_紫匙`
10	双色托桂	`10_shuangse_tuogui`	`10_双色托桂`

为什么禁用中文和空格：Python 的 `os.path` 能处理中文，但许多第三方工具（含部分版本的 OpenCV、LabelImg、云平台）在中文路径下会静默失败或报编码错误。空格更麻烦——脚本拼接路径时会被截断。**这是踩过无数次坑之后的经验，不是洁癖。**

为什么加两位序号：YOLO 分类器按**文件夹名字母序**给类别编号（详见第十一章 11.2 节）。加两位序号后，字母序即序号序，索引稳定可预判；若直接用拼音无序号，新增一类可能插在中间，导致所有类别编号后移，原有的预测结果全部错位。

4.3 单张图片文件命名

格式：`{类别文件夹名}_三位序号.jpg`

示例：

代码

```
01_huangse_pingban_001.jpg
01_huangse_pingban_002.jpg
02_baise_guanban_001.jpg
```

序号从 001 连续编到该类最后一张，**中间不得跳号、不得重名**。重名会导致拷贝时覆盖，是最隐蔽的数据丢失方式。

五、拍摄规范

5.1 设备与相机设置

使用手机**后置主摄**（不要用前置，前置分辨率低且普遍不写入 GPS）。拍摄前完成以下设置：

设置项	要求	位置（以主流安卓为例）
定位权限	相机 App 允许"使用期间获取位置"	设置 → 应用 → 相机 → 权限 → 位置
记录位置信息	打开"保存位置信息"/"地理位置"开关	相机 → 设置（齿轮）→ 位置信息
照片格式	JPG（若为 HEIC，需后续转换）	相机 → 设置 → 格式

设置项	要求	位置（以主流安卓为例）
分辨率	最高或次高（不低于 1200 万像素）	相机 → 设置 → 分辨率
网格线	建议打开，辅助构图	相机 → 设置 → 网格
美颜 / 滤镜 / 水印	**全部关闭**	相机 → 设置
闪光灯	自动或关闭，禁用强制闪光	拍摄界面

iPhone 用户：设置 → 隐私与安全性 → 定位服务 → 相机 → 选择"使用 App 期间"；相机 App 内开启"保留设置 → 位置信息"。

5.2 经纬度必须保留（本任务最容易被破坏的环节）

GPS 信息写在照片的 EXIF 元数据里。以下操作会**永久抹除** EXIF，且不可恢复：

操作	后果	说明
微信发送原图	**EXIF 丢失**	微信会重新编码，即使勾选"原图"也会剥离部分元数据
QQ 发送图片	**EXIF 丢失**	同上
用截图代替拍照	**无 GPS**	截图不继承原图 EXIF
用"照片压缩"类小程序	**EXIF 丢失**	重编码时未写回元数据
部分格式转换工具	**EXIF 丢失**	需确认工具支持"保留元数据"
用图片编辑软件裁剪	可能丢失	多数编辑器会重写 EXIF，裁剪后需复查

正确的传输方式（任选其一）：

1. **数据线**：手机连接电脑，直接复制 `DCIM/Camera/` 下的原文件。
2. **AirDrop / 隔空投送**（iPhone 到 Mac）。
3. **网盘原图上传**：百度网盘、阿里云盘等选择"原图/原画质"上传，再下载。
4. **邮件附件**：以"附件"形式发送，不要插入正文。

每批照片转运后，必须抽查验证 GPS 是否还在。验证方法见附录 B。

> 实务经验：建议**拍摄当天就完成一次"原图备份 + GPS 抽检"**。等到一周后再发现 GPS 全丢，只能重新跑一趟展区。

5.3 构图与拍摄角度

同一类别下的照片，必须在**视角、距离、开放程度**三个维度上做变化。只有一张正面照，模型学不到这一类的稳定特征。

每一类建议覆盖以下六种拍摄位：

序号	视角	目的	说明
1	正面平视	花型轮廓	镜头与花心等高，主体占画面 1/2
2	45° 斜俯视	花瓣层次	最常用的角度，能看到花瓣排列
3	正俯视	花心与瓣序	从上往下拍，呈放射状
4	侧面 90°	花托与花梗	体现花的立体厚度
5	近距特写	瓣端形态	体现平瓣/匙瓣/管瓣的差异
6	整株环境	植株比例	带背景，体现株高与叶形

同时，每类应包含**不同开放程度**的样本：花蕾期、初开期、盛开期。这能让模型对同一品种在不同阶段的形态变化更鲁棒，但**必须记在同一类别下**，不得单独设类。

构图约束：

- 主体（花头）占画面面积不少于 **1/3**，建议 1/2。
- 背景尽量简洁。可用纯色纸板、深色布或利用天空作背景；避免杂乱的游客、栏杆、其他花丛。
- 顺光或柔光拍摄。避开正午顶光（易过曝）与强烈逆光（花瓣变黑）。
- 画面保持水平，花梗尽量竖直。

- 同一类别内的照片，**背景不要完全一致**。若全用同一块白纸，模型可能学到"白背景 = 这个类别"，换环境就失效。

5.4 拍摄数量分配建议

以 10 类、每类 45 张为例（总计 450 张）：

拍摄位	每类张数	小计
正面平视	8	80
45° 斜俯视	10	100
正俯视	6	60
侧面 90°	6	60
近距特写	8	80
整株环境	7	70
合计	**45**	**450**

同一类别建议拍摄 3—5 株不同的花，而不是对着一株猛拍 45 张。单株过多会导致样本多样性不足，模型学到的是"这一株长什么样"，而不是"这一类长什么样"。

3 人分组时，可按下表分工拍摄：

成员	负责类别	负责拍摄位
甲	第 1—4 类	正面平视、45° 斜俯视
乙	第 4—7 类	正俯视、侧面 90°
丙	第 7—10 类	近距特写、整株环境

相邻类别由两人共同负责，便于交叉复核分类是否准确。

5.5 禁止事项

1. 禁止使用网络下载图片、他人拍摄图片。

2. 禁止用 AI 生成图片。
3. 禁止对同一张照片做旋转、翻转、调色后当作多张使用。
4. 禁止截图代替拍照。
5. 禁止拍摄非菊花主体（如展牌、游客、建筑）充数。
6. 禁止篡改或伪造 GPS 坐标。

六、数据集组织结构

6.1 压缩包顶层结构

提交物是一个 ZIP 压缩包。解压后应直接看到以下内容（**不要有多余的嵌套层**）：

代码

```
kaifeng_chrysanthemum.zip
├── train/                          # 训练集
│   ├── 01_huangse_pingban/
│   │   ├── 01_huangse_pingban_001.jpg
│   │   ├── 01_huangse_pingban_002.jpg
│   │   └── ...
│   ├── 02_baise_guanban/
│   └── ...（共 10 个类别文件夹）
├── val/                            # 验证集，结构与 train 完全相同
│   ├── 01_huangse_pingban/
│   ├── 02_baise_guanban/
│   └── ...
├── test/                           # 测试集，结构与 train 完全相同
│   ├── 01_huangse_pingban/
│   └── ...
├── labels.csv                      # 元数据表（见第八章）
└── README.md                       # 说明文档（见第十五章）
```

三个硬性约束：

1. `train`、`val`、`test` 三个目录下**必须都有全部 10 个类别文件夹**，哪怕某个类别的验证集只有 5 张，也要建文件夹并放进去。空文件夹会导致类别索引错

位。

2. 三个子集的**类别文件夹名必须完全一致**，一个字都不能差。`01_huangse_pingban` 与 `01_huangse_pingpan` 会被识别成两个不同类别。
3. 图片**直接**放在类别文件夹下，**不得再套一层**（例如不要写成 `train/01_xxx/images/xxx.jpg`）。

6.2 为什么采用这个结构

这个结构同时是 PyTorch `ImageFolder` 与 Ultralytics YOLO **分类任务**的默认约定，也是学术界与工业界图像分类任务的通用格式。

YOLO26 分类任务的数据集不使用 YAML 配置文件，目录结构本身就是配置。按此组织后，加载与训练只需一行：

Python 代码

```
from ultralytics import YOLO

model = YOLO("yolo26n-cls.pt")
model.train(data="kaifeng_chrysanthemum", epochs=50, imgsz=224)
```

框架会自动把 `train/` 下的每个文件夹名映射为一个类别、把其中的图片读成张量。**结构规范换来的是零成本的标注**——不需要单独写文件记录每张图属于哪类，目录结构本身就是标注。

6.3 完整目录树示例（含具体文件名）

代码

```
kaifeng_chrysanthemum.zip
│
├── train/
│   ├── 01_huangse_pingban/
│   │   ├── 01_huangse_pingban_001.jpg
│   │   ├── 01_huangse_pingban_002.jpg
│   │   ├── ...
```

代码

```
│   │   └── 01_huangse_pingban_036.jpg
│   ├── 02_baise_guanban/
│   │   ├── 02_baise_guanban_001.jpg
│   │   └── ...
│   ├── 03_lvse_jiban/
│   ├── 04_fense_lianzuo/
│   ├── 05_hongse_qiuxing/
│   ├── 06_fuse_feiwu/
│   ├── 07_huangse_guanban/
│   ├── 08_baise_pingban/
│   ├── 09_zise_chiban/
│   └── 10_shuangse_tuogui/
│
├── val/
│   ├── 01_huangse_pingban/
│   │   ├── 01_huangse_pingban_037.jpg
│   │   └── 01_huangse_pingban_038.jpg
│   ├── ... （同样 10 个类别）
│
├── test/
│   ├── 01_huangse_pingban/
│   │   ├── 01_huangse_pingban_039.jpg
│   │   ├── 01_huangse_pingban_040.jpg
│   │   └── 01_huangse_pingban_041.jpg
│   ├── ... （同样 10 个类别）
│
├── labels.csv
└── README.md
```

注意：编号在三个子集之间是**连续**的（train 到 036，val 从 037 开始，test 从 039 开始），不重新从 001 计数。这样能一眼看出某张图原本属于哪一类的第几张，便于追溯。

七、训练集、验证集、测试集的划分

7.1 划分比例与数量核算

推荐 8 ： 1 ： 1。以 10 类、每类 45 张为例：

集合	每类张数	各类总计	用途

集合	每类张数	各类总计	用途
train	36	360	训练模型、更新参数
val	4	40	调超参数、选择最优轮次
test	5	50	最终验收，只用一次
合计	**45**	**450**	—

若每类只能拍到 30 张，则按 24 : 3 : 3 划分，测试集仅 3 张——此时单张误判就会让准确率跳动 33 个百分点，**指标不可信**。这是建议每类拍到 40 张以上的实际原因。

硬性下限：每个类别的测试集**不少于 5 张**。

7.2 防泄漏原则（本任务最关键的规则）

同一次拍摄、同一株花的多张照片，必须整体进入同一个集合。

这条规则的道理很直接：如果同一朵花的正面照在训练集、侧面照在测试集，模型在训练时已经"见过"这朵花，测试时自然答对——但这不是学会了分类，而是**背下了答案**。此时测试准确率会虚高十几个百分点，而模型换到真实场景就崩掉。学术界称之为**数据泄漏（data leakage）**，是图像数据集构建中最常见、最致命的错误。

错误做法（按图片随机打乱后切割）：

代码

```
450 张照片 → random.shuffle() → 前 360 张进 train，接着 40 张进 val，最后 50 张进 test
```

这样每个集合里都有同一株花的不同角度，泄漏不可避免。

正确做法（按植株分组划分）：

1. 拍摄时记录每张照片对应的**植株编号**（如 `P01`、`P02`，可写在手机备忘录，或按拍摄顺序分批）。

2. 划分时以"植株"为单位分配集合，而不是以"图片"为单位。
3. 例如某类有 45 张照片，来自 5 株（每株 9 张）：P01—P04 进 train（36 张），P05 的 9 张拆分——前 4 张进 val、后 5 张进 test。

简化处理方式（如果没记植株编号）：**按拍摄时间分批**。拍摄时以"一株拍完再换下一株"为习惯，那么照片的连续序号天然对应植株，按序号区间切分即可。

> 若确实无法区分植株，退而求其次：**按时间顺序整体切分**（前 80% 进 train，中间 10% 进 val，最后 10% 进 test），不做随机打乱。这样至少保证相邻时间拍摄的照片（很可能是同一株）落在同一集合。这比随机打乱安全得多。

7.3 划分后必须自查

检查项	通过标准
是否有同一图片同时出现在两个集合	文件名全局唯一，无重复
三个集合的类别文件夹是否齐全	各 10 个，名称一致
测试集每类是否 ≥ 5 张	全部满足
各集合总数是否与预期一致	360 / 40 / 50（按 450 张计）

八、元数据表 labels.csv

压缩包根目录下必须有一份 `labels.csv`，记录每张图的完整信息。这是本任务的"数据说明书"，也是评分时核验 GPS 的唯一依据。

8.1 字段定义

字段名	类型	是否必填	说明
`filename`	字符串	必填	图片文件名，含扩展名
`split`	字符串	必填	`train` / `val` / `test`

字段名	类型	是否必填	说明
category	字符串	必填	类别文件夹名，如 01_huangse_pingban
plant_id	字符串	必填	植株编号，如 P01；用于核验划分是否泄漏
latitude	浮点	必填	纬度，十进制度，保留 6 位小数
longitude	浮点	必填	经度，十进制度，保留 6 位小数
altitude	浮点	选填	海拔（米）
datetime	字符串	必填	拍摄时间，格式 YYYY-MM-DD HH:MM:SS
device	字符串	选填	拍摄设备型号

8.2 样例

CSV 样例

```
filename,split,category,plant_id,latitude,longitude,altitude,datetime,device
01_huangse_pingban_001.jpg,train,01_huangse_pingban,P01,34.797312,114.307456,72.5,2026-11-08 09:14:22,Redmi K70
01_huangse_pingban_002.jpg,train,01_huangse_pingban,P01,34.797318,114.307461,72.4,2026-11-08 09:14:45,Redmi K70
01_huangse_pingban_037.jpg,val,01_huangse_pingban,P05,34.797652,114.308120,72.6,2026-11-08 10:22:10,Redmi K70
```

8.3 生成方式

不建议手工填写。用 Python 读取每张图的 EXIF 自动生成，脚本见附录 B。手工填 450 行既耗时又必然出错，且无法保证经纬度真实。

九、执行步骤

整个任务分七个阶段，建议投入 4—5 周。时间安排可按实际调整，但**阶段顺序不可颠倒**——尤其"先选类别再拍摄"这一步，不能拍完再想怎么分类。

阶段 0：准备（1 天）

1. **组建 3 人小组**，确定组名，填写第三章的分工表。
2. **选定展区与时间**。建议上午 9:00—11:00，光线柔和、游客较少。龙亭公园、清明上河园菊展规模较大。
3. **确定 10 个类别**，写成清单（可用 Excel）。清单格式见 4.2 节表格。
4. **与其他组沟通差异化方案**：确认本组的分类维度、展区、模型配置不与别组重复。
5. **检查手机设置**：定位权限、位置信息开关、照片格式、分辨率、关闭美颜与水印。**现场拍摄前必须验证一次 GPS 是否写入**（拍一张，立刻查属性）。
6. **准备辅助工具**：深色硬纸板一块（作背景）、软毛刷（清理花瓣灰尘，可选）、充电宝。
7. **建立本地工作目录**，按第六章结构预建空文件夹，并放入占位文件（如 `.gitkeep`），确保压包时不丢空目录。

阶段 1：采点与拍摄（2—3 天）

1. 到达展区后，**先定位、后拍摄**。打开手机地图确认定位已锁定（蓝点稳定）。
2. 按类别逐类拍摄。**一类拍完再换下一类**，避免来回穿插导致后期归类混乱。
3. 每类开头拍一张"类别标识照"（含展区品种标牌），单独存放在 `_raw/` 目录，**不进入数据集**，仅用于后期核对类别名与品种。这是防错的重要手段。
4. 按 5.3 节的六个拍摄位逐株拍完，再换下一株。
5. 每完成一类，在清单上打勾并记录张数。

阶段 2：原图归集与清洗（1 天）

1. 用**数据线或网盘原图**把照片拷到电脑的 `_raw/{类别名}/` 目录。**不要用微信传输。**
2. **抽检 GPS**：每类随机抽 3 张，右键 → 属性 → 详细信息 → 查看"GPS 纬度""GPS 经度"是否有值（见附录 B）。
3. **剔除废片**：模糊、过曝、主体过小、重复、拍到非菊花主体的，全部删除。
4. **去重**：按文件哈希值查重，删除完全相同的图片。
5. **格式统一**：HEIC 转 JPG，长边缩放到 2048 像素以内，质量 90。转换时**必须保留 EXIF**。

阶段 3：命名与归类（1 天）

1. 按 4.3 节规则批量重命名为 `{类别}_{序号}.jpg`。
2. 按第七章的防泄漏原则，确定每张图的 `plant_id` 与所属集合。
3. 复制（**不是移动**）到 `train/val/test` 对应的类别文件夹下。保留 `_raw/` 原目录作为备份，直到提交前再删除。
4. 核对三处：每个类别文件夹张数、三个子集类别是否齐全、文件名是否全局唯一。

阶段 4：划分复核（半天）

1. 统计各集合、各类别的图片数量，填入 7.3 节的核对表。
2. **逐类检查是否有泄漏**：同一 `plant_id` 是否跨越了不同集合。若有，调整。
3. 确认测试集每类 ≥ 5 张。

阶段 5：生成元数据与说明文档（半天）

1. 运行脚本生成 `labels.csv`（附录 B）。
2. 检查 `labels.csv` 行数是否等于图片总数。
3. 撰写 `README.md`，内容见第十五章。

阶段 6：环境配置与训练（2—3 天）

1. 按第十章配置 Python 环境、安装 Ultralytics、设置环境变量。
2. 按第十一章编写训练脚本，先跑 5 轮快速验证流程是否通。
3. 确认无误后正式训练（50—100 轮），保存训练日志。
4. 在测试集上评估，导出混淆矩阵与预测结果。

阶段 7：打包与自检（1 天）

1. 汇总代码、日志、图表到 `code/` 与 `results/` 目录。
2. 确认目录结构符合 6.1 节。
3. 压缩为 ZIP。**压缩包内不要包含上级文件夹**——选中 `train`、`val`、`test`、`labels.csv`、`README.md`、`code`、`results` 七项，右键压缩，而不是压缩整个工作目录。
4. 解压到临时目录，重跑一次训练脚本，确认可复现。
5. 按第十三章清单逐项自检。

十、环境配置与依赖安装

本节所有命令均在本组选定的工作目录下执行。**建议整个项目放在纯英文路**

径（如 `D:\chrysanthemum`），不要放在桌面或含中文、空格的目录下——训练框架的部分组件在中文路径下会静默失败。

10.1 硬件与系统要求

项目	最低要求	推荐	说明
操作系统	Windows 10 64 位 / Ubuntu 20.04	Windows 11 / Ubuntu 22.04	本项目两条路径都验证可行
内存	8 GB	16 GB 以上	数据加载占内存，8 GB 需把 `workers` 调小
显卡	集成显卡可跑	NVIDIA GTX 1650 及以上	有独显时训练快 5—10 倍
显存	—	4 GB 以上	224 输入 + `batch=32` 约需 2.5 GB
硬盘	5 GB 可用空间	20 GB	含数据集、环境、训练输出
Python	3.10	3.11 或 3.12	Ultralytics 支持的版本区间

> 没有独立显卡也能完成本任务，只是训练时间从 10 分钟延长到 40—60 分钟。**CPU 训练不影响评分**，只要 `device` 参数如实填写。

10.2 第一步：创建独立的虚拟环境

必须使用虚拟环境。直接装在系统 Python 里，将来装别的库极易冲突，且出问题难以排查。

方式一：venv（Python 自带，最轻量）

Windows 命令提示符或 PowerShell：

代码

```
cd /d D:\chrysanthemum
python -m venv .venv
```

代码

```
.venv\Scripts\activate
```

Linux / macOS：

代码

```
cd ~/chrysanthemum
python3 -m venv .venv
source .venv/bin/activate
```

激活成功的标志：命令行提示符前出现 `(.venv)`。

方式二：Conda（推荐，隔离更彻底）

代码

```
conda create -n chrys python=3.11 -y
conda activate chrys
```

常见问题：PowerShell 提示"无法加载文件 activate.ps1，因为在此系统上禁止运行脚本"。解决方法是先执行 `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`，输入 `Y` 确认，再重新激活。

10.3 第二步：安装 PyTorch

先装 PyTorch，再装 Ultralytics。顺序反了会装错 CUDA 版本。

有 NVIDIA 显卡（CUDA 12.1 示例）：

代码

```
pip install torch torchvision --index-url
https://download.pytorch.org/whl/cu121
```

只有 CPU：

代码

```
pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
```

验证安装是否正常：

Python 代码

Python 代码

```
import torch
print("PyTorch 版本:", torch.__version__)
print("CUDA 是否可用:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("显卡型号:", torch.cuda.get_device_name(0))
```

预期输出（有显卡时）：`CUDA 是否可用: True`。若为 `False` 但你有显卡，说明装成了 CPU 版，需卸载重装：

代码

```
pip uninstall torch torchvision -y
pip install torch torchvision --index-url
https://download.pytorch.org/whl/cu121
```

10.4 第三步：安装 Ultralytics

代码

```
pip install ultralytics
```

国内网络较慢时用镜像源：

代码

```
pip install ultralytics -i https://pypi.tuna.tsinghua.edu.cn/simple
```

验证安装：

代码

```
yolo checks
```

该命令会打印 Python 版本、PyTorch 版本、CUDA 状态、CPU 信息，最后显示 `Ultralytics` 版本号。**截图保存这个输出**，作为环境配置的证据写入 README。

也可以直接用 Python 验证：

Python 代码

```
import ultralytics
print("Ultralytics 版本:", ultralytics.__version__)
from ultralytics import YOLO
print("YOLO 导入成功")
```

10.5 第四步：设置环境变量

环境变量决定框架把配置、缓存、权重放在哪里。默认位置在用户目录下，多组共用一台电脑时容易互相覆盖，**建议每组设置独立目录**。

Windows PowerShell（当前会话生效）：

```powershell
$env:YOLO_CONFIG_DIR = "D:\chrysanthemum\.yolo_config"
$env:CUDA_VISIBLE_DEVICES = "0"
$env:OMP_NUM_THREADS = "4"
```

Windows 命令提示符（当前会话生效）：

```bat
set YOLO_CONFIG_DIR=D:\chrysanthemum\.yolo_config
set CUDA_VISIBLE_DEVICES=0
set OMP_NUM_THREADS=4
```

Windows 永久生效（新开终端后有效）：

```bat
setx YOLO_CONFIG_DIR "D:\chrysanthemum\.yolo_config"
setx OMP_NUM_THREADS "4"
```

Linux / macOS：

```bash
export YOLO_CONFIG_DIR=~/chrysanthemum/.yolo_config
export CUDA_VISIBLE_DEVICES=0
export OMP_NUM_THREADS=4
```

写入 `~/.bashrc` 可永久生效：

```bash
echo 'export YOLO_CONFIG_DIR=~/chrysanthemum/.yolo_config' >> ~/.bashrc
source ~/.bashrc
```

三个变量的作用与必要性：

变量	作用	是否必须	说明

变量	作用	是否必须	说明
YOLO_CONFIG_DIR	指定框架配置文件与缓存目录	建议设置	多组共用电脑时**必须**各自独立，否则配置互相覆盖
CUDA_VISIBLE_DEVICES	限制可见的显卡编号	多卡机器必须	单卡可省略；"0" 表示只用第 1 块卡
OMP_NUM_THREADS	限制 CPU 并行线程数	Windows 建议	设得过大反而变慢，一般设为物理核心数的一半

> 关于 YOLO_CONFIG_DIR 的准确性说明：该变量在 Ultralytics 生态中广泛使用，但**并未全部出现在官方 Train 文档的正式参数表中**。不同版本行为可能略有差异，设置后请用 yolo checks 确认输出中的配置路径确实指向你设定的目录；若未生效，可改为在脚本开头用 os.environ 赋值（见 11.1 节代码）。另一种常见说法是设置 USE_LIBUV=0 解决 Windows 多卡报错。**官方文档已明确说明该变量无法解决此问题**（多进程启动不走该代码路径）。Windows 上请直接使用单卡 device=0，不要尝试多卡。

10.6 第五步：目录准备

在工作目录下建立以下结构，把数据集放进去：

代码

```
D:\chrysanthemum\
├── datasets\
│   └── kaifeng_chrysanthemum\
│       ├── train\
│       ├── val\
│       └── test\
├── code\
│   ├── train.py
│   ├── evaluate.py
│   └── predict.py
├── results\
├── .yolo_config\
└── .venv\
```

10.7 环境配置常见问题

报错信息	原因	解决
`ModuleNotFoundError: No module named 'ultralytics'`	虚拟环境未激活，或装到了别的环境	确认提示符有 `(.venv)`；`pip list` 查看是否有 ultralytics
`torch.cuda.is_available()` 返回 `False`	装的是 CPU 版 torch，或显卡驱动过旧	卸载重装 CUDA 版；更新 NVIDIA 驱动
`CUDA out of memory`	显存不足	减小 `batch`（32→16→8）；减小 `imgsz`
`RuntimeError: use_libuv was requested but PyTorch was built without libuv`	Windows 多卡训练	改用单卡 `device=0`，不要设多卡
训练卡住不动，进度条不动	`workers` 过多或路径含中文	设 `workers=0`；改用纯英文路径
`PermissionError` 写入配置目录失败	`.yolo_config` 无写权限	换目录，或以管理员身份运行
首次运行卡在下载权重	从 GitHub 拉取 `yolo26n-cls.pt` 慢	耐心等待；或手动下载后放到工作目录
`FileNotFoundError: No such file or directory`	路径反斜杠被当转义符	用原始字符串 `r"D:\..."` 或正斜杠 `"D:/..."`

完整的环境排错流程见附录 A。

十一、YOLO26 分类模型训练

11.1 先理解一件事：分类任务的数据集不需要 YAML

YOLO 的**检测任务**需要 `data.yaml` 声明类别名与图片路径；但**分类任务不用**——目录结构就是配置。

项目	检测任务	分类任务
配置文件	必须有 `data.yaml`	**不需要**，传目录路径即可

项目	检测任务	分类任务
标签形式	每张图配 `.txt` 标注框	**文件夹名即标签**
`data` 参数	指向 `.yaml` 文件	指向**包含 train/val 的根目录**
验证集目录名	YAML 中指定	自动查找 `val/`，找不到则依次尝试 `validation/`、`valid/`、`test/`

我们的数据集已经是 `train/val/test` 结构，**直接可用，无需任何转换**。这是第六章强调目录结构规范的实际收益。

11.2 类别索引按文件夹名字母序生成

YOLO 分类器按**文件夹名的字母序**给类别编号，从 0 开始。以我们的命名规则为例：

文件夹名	类别索引
`01_huangse_pingban`	0
`02_baise_guanban`	1
`03_lvse_jiban`	2
...	...
`10_shuangse_tuogui`	9

因为加了两位序号，字母序恰好等于序号序，索引稳定。**这也是 4.2 节要求加序号的原因**。

关键纪律：永远不要硬编码类别索引。必须用 `model.names` 做映射：

Python 代码

```
from ultralytics import YOLO

model = YOLO("runs/classify/chrys_yolo26n/weights/best.pt")
print(model.names)
# {0: '01_huangse_pingban', 1: '02_baise_guanban', ...}

result = model("test.jpg")[0]
```

Python 代码

```
class_idx = result.probs.top1          # 整数索引
class_name = model.names[class_idx]    # 用 names 映射成名称
print(f"预测为 {class_name}，置信度 {result.probs.top1conf:.4f}")
```

如果直接把索引 0 当作"黄色平瓣"写死在代码里，将来插入一个新类别导致排序变化，所有结果都会错位，而且**程序不报错**。

11.3 训练脚本（完整版）

在工作目录 `code/` 下新建 `train.py`。以下代码可直接运行，**改掉开头的路径配置即可**。

Python 代码

```
import os
from pathlib import Path

# ============ 环境变量：必须在导入 ultralytics 之前设置 ============
os.environ.setdefault("YOLO_CONFIG_DIR", str(Path(__file__).parent.parent /
".yolo_config"))
os.environ.setdefault("OMP_NUM_THREADS", "4")

from ultralytics import YOLO
import torch


# ============ 本组配置：按实际情况修改 ============
CONFIG = {
    "data_root":  r"D:\chrysanthemum\datasets\kaifeng_chrysanthemum",
    "model":      "yolo26n-cls.pt",    # 型号：n / s / m，按差异化要求选择
    "epochs":     50,                  # 训练轮数
    "imgsz":      224,                 # 输入尺寸：224 / 320 / 448
    "batch":      32,                  # 批大小，显存不足时减半
    "device":     0,                   # 0 表示第 1 块显卡；无显卡填 "cpu"
    "workers":    2,                   # 数据加载线程；Windows 建议 0—4
    "patience":   15,                  # 验证指标连续 N 轮不提升则早停
    "optimizer":  "auto",              # auto / SGD / AdamW
    "lr0":        0.001,               # 初始学习率（optimizer=auto 时被忽略）
    "dropout":    0.0,                 # 分类任务专用正则化，过拟合时可设 0.2
    "freeze":     None,                # 冻结前 N 层；None 表示全量微调
    "seed":       42,                  # 随机种子，保证可复现
```

Python 代码

```
    "pretrained": True,               # 使用 ImageNet 预训练权重
    "project":    r"D:\chrysanthemum\results\runs",   # 输出根目录
    "name":       "chrys_yolo26n",    # 本次运行名，输出落在 project/name/
    "exist_ok":   True,               # 允许覆盖同名目录
    "plots":      True,               # 生成训练曲线与混淆矩阵图
    "cache":      False,              # 数据集小可设 True 加速
}


def check_device(cfg):
    """训练前确认设备可用，避免跑到一半报错"""
    if str(cfg["device"]).lower() == "cpu":
        print("[设备] 使用 CPU 训练，速度较慢但可正常完成")
        return
    if not torch.cuda.is_available():
        print("[警告] 未检测到可用 CUDA，自动切换为 CPU")
        cfg["device"] = "cpu"
    else:
        print(f"[设备] {torch.cuda.get_device_name(cfg['device'])}"
              f"，显存
{torch.cuda.get_device_properties(cfg['device']).total_memory /
1024**3:.1f} GB")


def main():
    check_device(CONFIG)

    data_root = Path(CONFIG["data_root"])
    if not data_root.is_dir():
        raise FileNotFoundError(f"数据集目录不存在：{data_root}")

    # 训练前确认目录结构（三件套齐全）
    for sub in ("train", "val"):
        p = data_root / sub
        if not p.is_dir():
            raise FileNotFoundError(f"缺少 {sub} 目录：{p}")
        classes = sorted([d.name for d in p.iterdir() if d.is_dir()])
        print(f"[检查] {sub}/ 下 {len(classes)} 个类别：{classes[:3]}...")

    # 加载预训练模型
    model = YOLO(CONFIG["model"])
    print(f"[模型] {CONFIG['model']} 已加载")

    # 开始训练
    results = model.train(
```

Python 代码

```
        data=str(data_root),
        epochs=CONFIG["epochs"],
        imgsz=CONFIG["imgsz"],
        batch=CONFIG["batch"],
        device=CONFIG["device"],
        workers=CONFIG["workers"],
        patience=CONFIG["patience"],
        optimizer=CONFIG["optimizer"],
        lr0=CONFIG["lr0"],
        dropout=CONFIG["dropout"],
        freeze=CONFIG["freeze"],
        seed=CONFIG["seed"],
        pretrained=CONFIG["pretrained"],
        project=CONFIG["project"],
        name=CONFIG["name"],
        exist_ok=CONFIG["exist_ok"],
        plots=CONFIG["plots"],
        cache=CONFIG["cache"],
        val=True,
        save=True,
        verbose=True,
    )

    print("\n=== 训练完成 ===")
    print(f"最优权重: {model.trainer.best}")
    print(f"最后一轮: {model.trainer.last}")
    return results


# Windows 上多进程训练必须放在 __main__ 保护块内，否则会反复重启进程
if __name__ == "__main__":
    main()
```

`if __name__ == "__main__":` 这一行不能删。 Windows 用 `spawn` 方式启动子进程，没有这个保护块，数据加载器的子进程会重新执行整个脚本，导致无限递归启动，最终报内存错误。这是 Windows 上训练"卡死"或"莫名退出"的最常见原因。

11.4 训练参数的取舍

不必每个参数都调，但下面六个要理解：

参数	影响	建议取值	说明
`model`	网络规模	按差异化要求	`n`（2.8M 参数）最快，`m`（11.6M）最强但慢 3 倍
`imgsz`	输入分辨率	224 起步	224 是 ImageNet 预训练尺寸，最省时；花瓣细节难分时可升到 320 或 448
`epochs`	训练轮数	50—100	本项目数据量小，50 轮通常足够；配合 `patience` 早停可避免无效训练
`batch`	批大小	32，显存不足砍半	太小梯度噪声大，太大泛化差
`patience`	早停耐心值	15	验证指标 15 轮不提升就停止，防止过拟合浪费时间
`freeze`	冻结层数	`None` 或 4	数据少时冻结主干前几层可减少过拟合，也是差异化手段

关于 `lr0` 的一个陷阱：当 `optimizer="auto"` 时，框架会自己选择优化器并设定学习率，**你传入的 `lr0` 会被忽略**。想手动控制学习率，必须显式指定优化器：

Python 代码

```
optimizer="AdamW", lr0=0.001
```

11.5 读懂训练输出

训练开始后，控制台会逐轮打印。典型输出：

代码

```
Starting training for 50 epochs...
      Epoch    GPU_mem       loss  top1_acc  top5_acc
        1/50      1.82G      1.836     0.312     0.788
        2/50      1.82G      1.402     0.478     0.891
        3/50      1.81G      1.105     0.583     0.942
        ...
       50/50      1.81G      0.213     0.917     0.997
```

四个指标的含义：

指标	含义	健康表现
`loss`	训练损失，越低说明拟合越好	逐轮下降，最后趋于平稳

指标	含义	健康表现
top1_acc	首选类猜对的比率（**这是主指标**）	稳步上升，通常能到 0.85 以上
top5_acc	正确答案在前 5 个候选里的比率	10 类任务中会很快接近 1.0，参考价值有限
GPU_mem	显存占用	稳定在某一数值，若持续上涨则有内存泄漏

判断训练是否正常，看三条曲线（训练结束后自动保存在 results/runs/chrys_yolo26n/ 下）：

1. **results.png**：总览图，含 loss 与 accuracy 两条曲线。
2. **confusion_matrix.png**：混淆矩阵，看哪两类容易混。
3. **val_batch0_pred.jpg**：验证集样例的预测结果可视化。

三种异常及处理：

现象	判断	处理
训练 loss 下降，但 val 准确率不涨甚至下降	过拟合	加 dropout=0.2；加数据增强；降 epochs；做 freeze
训练与验证准确率都停在很低水平（如 0.15）	欠拟合或数据问题	检查类别文件夹是否为空；提高 epochs；换更大模型
loss 变成 nan 或剧烈震荡	学习率过大或数据异常	降 lr0 到 1e-4；检查是否有损坏图片

11.6 训练后评估：三个集合分别测

训练结束后，best.pt 是在**验证集**上挑出来的。最终成绩要在**测试集**上取，且只取一次。

在工作目录 code/ 下新建 evaluate.py：

Python 代码

```
import os
```

Python 代码

```
import hashlib
import json
from pathlib import Path

os.environ.setdefault("YOLO_CONFIG_DIR", str(Path(__file__).parent.parent /
".yolo_config"))

from ultralytics import YOLO


BEST = r"D:\chrysanthemum\results\runs\chrys_yolo26n\weights\best.pt"
DATA = r"D:\chrysanthemum\datasets\kaifeng_chrysanthemum"

# 本组的模型配置（与 train.py 的 CONFIG 保持一致）
FINGERPRINT = {
    "model": "yolo26n-cls",
    "imgsz": 224,
    "freeze": None,
    "augment": "randaugment",
    "optimizer": "auto",
    "epochs": 50,
}


def main():
    model = YOLO(BEST)

    print("=== 类别映射 ===")
    for idx, name in model.names.items():
        print(f"  {idx}: {name}")

    print("\n=== 三个集合的准确率 ===")
    summary = {}
    for split in ("train", "val", "test"):
        metrics = model.val(data=DATA, split=split, imgsz=224, batch=32,
verbose=False)
        summary[split] = {
            "top1": round(float(metrics.top1), 4),
            "top5": round(float(metrics.top5), 4),
        }
        print(f"  {split:>5}: top1 = {metrics.top1:.4f}   top5 =
{metrics.top5:.4f}")

    # 保存结果
    out = Path(BEST).parent.parent / "eval_summary.json"
    out.write_text(json.dumps(summary, ensure_ascii=False, indent=2),
```

Python 代码

```
encoding="utf-8")
    print(f"\n 结果已保存：{out}")

    # 模型指纹（用于组间差异核查）
    s = json.dumps(FINGERPRINT, sort_keys=True, ensure_ascii=False)
    fp = hashlib.md5(s.encode()).hexdigest()[:8].upper()
    print(f"模型指纹：{fp}")
    print(f"  配置明细：{s}")


if __name__ == "__main__":
    main()
```

三份准确率的意义：

集合	用途	正常关系
train	训练时见过	最高，通常 0.95 以上
val	训练中用于选最优轮次	次之，比 train 低 3—8 个百分点
test	**最终验收，只看一次**	与 val 接近；若远低于 val，说明过拟合严重

若 `train` 与 `test` 的差距超过 15 个百分点，需要在报告里如实分析原因，不能只报告测试集数字。

11.7 单张图片预测

新建 `predict.py`，用于演示与答辩：

Python 代码

```
import os
from pathlib import Path

os.environ.setdefault("YOLO_CONFIG_DIR", str(Path(__file__).parent.parent /
".yolo_config"))

from ultralytics import YOLO

BEST = r"D:\chrysanthemum\results\runs\chrys_yolo26n\weights\best.pt"
```

Python 代码

```
model = YOLO(BEST)

# 预测单张或整个文件夹
results =
model(r"D:\chrysanthemum\datasets\kaifeng_chrysanthemum\test\01_huangse_pingban"
, save=True)

for r in results[:5]:
    idx = r.probs.top1
    conf = r.probs.top1conf
    name = model.names[idx]
    print(f"{Path(r.path).name:<40} → {name:<26} {conf:.4f}")
```

`save=True` 会把带预测标注的图片存到 `runs/classify/predict/` 下，可直接用于汇报 PPT。

11.8 断点续训

训练中断（断电、误关窗口、超时）时不必从头再来：

Python 代码

```
from ultralytics import YOLO

model =
YOLO(r"D:\chrysanthemum\results\runs\chrys_yolo26n\weights\last.pt")
model.train(resume=True)
```

`resume=True` 会恢复权重、优化器状态、学习率调度、已完成的轮数。**注意：**必须至少完成 1 个 epoch 才有 `last.pt` 可续。

十二、结果分析与可视化

训练跑完不等于任务结束。**能解释结果，才是这门课要的能力。**

12.1 必须产出的四张图

图	文件	说明

图	文件	说明
训练曲线	`results.png`	含 loss 与 accuracy 随轮次变化
混淆矩阵	`confusion_matrix.png`	10×10 网格，看哪两类容易混
归一化混淆矩阵	`confusion_matrix_normalized.png`	按行归一化，消除类别样本数差异的影响
预测样例	`val_batch0_pred.jpg`	直观看到模型在哪些图上出错

四张图都要放入 `results/` 目录，并在 README 与分析报告中引用。

12.2 怎么读混淆矩阵

混淆矩阵的行是**真实类别**，列是**预测类别**。对角线上的数字是预测正确的数量。

以 10 类任务为例，理想的混淆矩阵应该近似对角矩阵。若出现以下情况，说明有具体问题：

现象	含义	可能原因
非对角线上出现较大数字	两类互相混淆	这两类形态太像，或标注有误
某一行正确率特别低	该类整体识别差	该类样本太少，或拍摄角度单一
对角线上某类数字明显小于测试样本数	该类频繁被判成别的类	类别定义有重叠，或该类的图混入了其他类

必须做的一件事：把混淆最严重的那一对类别，各挑 5 张原图并列对比，写进报告。这是分析报告里最有价值的部分——它证明了你在**真正看数据**，而不是只抄一个准确率数字。

12.3 错误样本分析

从测试集里挑出被判错的图片（通常 5—10 张），逐张分析原因。常见原因分四类：

类别	具体表现	说明
标注错误	人看也觉得标错了	数据质量问题，应修正后重训
类别定义模糊	人看也分不清属于哪类	类别体系设计问题，需在报告中说明
拍摄质量差	模糊、过曝、主体过小	采集问题
真实困难样本	形态确实介于两类之间	模型能力边界，属正常现象

分析时**必须附上原图编号**，便于核查。空泛地写"模型还有提升空间"不得分。

12.4 建议做的对比实验（加分）

差异化要求使得组间结果可以横向比较。本组内部也可以做一组对照，验证某个参数的作用：

对比项	做法	想验证什么
输入尺寸	224 vs 320	提高分辨率是否值得多花的时间
微调深度	全量 vs 冻结前 4 层	小数据集上冻结是否能缓解过拟合
网络规模	`yolo26n-cls` vs `yolo26s-cls`	参数量翻倍带来多少精度提升

做一组即可，在报告中给出两组的三份准确率对照表，并给出你的解释。**这是从"完成任务"到"做出研究"的分界。**

十三、提交前自检清单

打印此表逐项打勾，**全部为"是"才可提交**。

序号	检查项	是/否
1	已按 3 人一组完成，分工表已写入 README	
2	类别数 ≥ 10，且各类形态差异可辨	

序号	检查项	是/否
3	每类 ≥ 30 张，测试集每类 ≥ 5 张	
4	全部为本组在开封现场拍摄，无网络图片	
5	每张图片均含 GPS 经纬度，抽检 10 张全部通过	
6	`labels.csv` 行数 = 图片总张数	
7	`labels.csv` 中经纬度与图片 EXIF 一致	
8	经纬度落在开封范围（约 34.7—34.9° N，114.2—114.4° E）	
9	压缩包解压后直接是 train/val/test，无多余嵌套	
10	三个子集均含全部 10 个类别文件夹	
11	三个子集的类别文件夹名完全一致	
12	图片直接位于类别文件夹下，无中间层	
13	文件名全局唯一，无重名、无中文、无空格	
14	同一植株的照片未跨集合（无数据泄漏）	
15	组间数据集相似度已自查，结果 < 70%（附录 C 脚本）	
16	`train.py`、`evaluate.py`、`predict.py` 三个脚本齐全	
17	训练脚本有 `if __name__ == "__main__":` 保护块	
18	`train.py` 顶部有 CONFIG 字典，超参集中可改	
19	环境变量设置方式已写入 README（含具体命令）	
20	`yolo checks` 的输出已截图保存	
21	训练日志（控制台输出）已保存为 `train_log.txt`	
22	四张图表齐全：`results.png`、`confusion_matrix.png`、`confusion_matrix_normalized.png`、`val_batch0_pred.jpg`	
23	三个集合的 top1 准确率均已记录到 `eval_summary.json`	
24	混淆矩阵中易混的两类已做原图对比分析	
25	模型指纹已计算并登记，与同班其他组不同	

序号	检查项	是/否
26	在**另一台电脑**上按 README 重跑，能复现结果	
27	`README.md` 含分工、类别、环境、训练、结果、局限六部分	
28	压缩包命名符合第十五章规范	

> 第 26 项是"可复现"的硬检验。**在自己电脑上能跑不算通过，要换一台干净环境能跑通才算。** 建议组内互相验证：甲组的代码交给乙组跑一遍。

十四、常见错误与避坑指南

14.1 数据集相关

错误现象	根本原因	后果	正确做法
所有图都没有 GPS	用了微信传图	无法核验采集真实性，该项零分	数据线 / 网盘原图
部分图无 GPS	室内拍摄或定位未锁定	完整率不足 100%	拍前确认定位已锁定
类别数变成 20	文件夹名大小写或拼写不一致	训练时报错或索引错位	统一命名，用脚本批量建目录
某子集类别数不足 10	漏建文件夹或某类样本太少	训练中断	三个子集补全 10 个文件夹
测试准确率 99% 但实际识别很差	数据泄漏	结论完全错误	按植株分组划分
压缩包解压后多一层文件夹	压缩了整个工作目录	加载路径错	选中目录内容压缩
图片全是同一株同一角度	拍得省事	模型过拟合到单株	每类 3—5 株 × 6 个视角
换背景后识别失败	背景单一，模型学了背景	泛化差	背景多样化
组间数据集被判雷同	直接复制他人照片	该维度零分	各组独立拍摄，选不同展区

14.2 环境与训练相关

错误现象	根本原因	正确做法
`No module named 'ultralytics'`	虚拟环境未激活	确认提示符有 `(.venv)`，重新 `activate`
`torch.cuda.is_available()` 为 `False`	装成 CPU 版 torch	卸载重装 CUDA 版，检查显卡驱动
`CUDA out of memory`	显存不足	减小 `batch`（32→16→8）或 `imgsz`（224→192）
训练一启动就退出，无明显报错	Windows 缺 `__main__` 保护块	把训练代码放进 `if __name__ == "__main__":`
训练卡住，进度条不动	`workers` 过多	设 `workers=0` 或 `2`
路径报 `unicode escape` 错	反斜杠被当转义符	用 `r"D:\path"` 或 `"D:/path"`
首次训练卡在下载	从 GitHub 拉权重慢	耐心等，或手动下载 `yolo26n-cls.pt` 放工作目录
`lr0` 设了但没生效	用了 `optimizer="auto"`	显式指定 `optimizer="AdamW"`
训练 loss 变 `nan`	学习率过大或图片损坏	降 `lr0` 到 1e-4；用附录 B 脚本检查图片能否打开
找不到 `best.pt`	运行名与 `project` 拼错	到 `results/runs/` 下逐层找
val 准确率不动	类别文件夹为空，或图片全是同一张	统计每类张数，检查是否有重复图
换了电脑跑不出同样结果	未固定 `seed`，或环境版本不同	设 `seed=42`；README 写明 ultralytics 与 torch 版本
模型指纹与别组相同	四项配置全一致	按 3.3 节调整至少三项

十五、提交要求

15.1 压缩包命名

格式：`{学号}_{姓名}_菊花数据集_{类别数}类_{总张数}张.zip`

3 人小组由排在第一位的同学学号命名，其余成员学号写在 README 分工表中。

示例：`2025001_张三_菊花数据集_10 类_452 张.zip`

15.2 压缩包内容

必须包含且仅包含以下七项（顶层）：

序号	内容	说明
1	`train/`	训练集目录
2	`val/`	验证集目录
3	`test/`	测试集目录
4	`labels.csv`	元数据表
5	`README.md`	说明文档
6	`code/`	全部代码脚本
7	`results/`	训练日志与图表

`code/` 目录必须包含：

代码

```
code/
├── train.py              # 训练脚本（含 CONFIG 配置块）
├── evaluate.py           # 三集合评估脚本
├── predict.py            # 单张/批量预测脚本
├── build_labels.py       # 生成 labels.csv（附录 B）
├── check_dataset.py      # 数据集结构自查
└── requirements.txt      # 依赖清单
```

`requirements.txt` 由下面命令生成，**必须用固定版本号**，否则别人装到新版

本可能行为不同：

```
代码
pip freeze > code/requirements.txt
```

`results/` 目录必须包含：

```
代码
results/
├── runs/
│   └── chrys_yolo26n/
│       ├── weights/
│       │   └──── best.pt              # 最优权重
│       ├──── results.png              # 训练曲线
│       ├── confusion_matrix.png
│       ├── confusion_matrix_normalized.png
│       └──── args.yaml                # 训练参数快照
├──── eval_summary.json                # 三集合准确率
├──── train_log.txt                    # 控制台输出
└──── error_cases/                     # 错误样本原图（5—10 张）
```

`best.pt` 通常 3—6 MB，可随包提交；若超过 20 MB（如 `yolo26m-cls`），可单独上传并在 README 给出下载链接，但**必须在包内保留 `eval_summary.json` 与全部图表**。

15.3 README.md 内容要求（六部分）

用 Markdown 编写，缺任一部分扣分：

第一部分　小组信息与分工三位成员姓名、学号；按 3.1 节表格写明每人职责与交付物。

第二部分　数据集概述与类别说明类别数、总张数、采集时间、采集地点；类别说明表（文件夹名、中文类别名、划分依据、样本数量）；按植株与拍摄位的分布统计。

第三部分　环境配置操作系统与版本；Python 版本；`torch`、`torchvision`、`ultralytics` 版本号；是否使用 GPU 及型号；**环境变量设置的具体命令**（照抄你

实际执行的那几条）；`yolo checks` 输出截图。

第四部分　训练过程模型型号与选择理由；完整超参表（至少含 `model`、`imgsz`、`batch`、`epochs`、`optimizer`、`lr0`、`freeze`、`seed`）；训练耗时；训练曲线截图与解读；**模型指纹**（6 项配置 + 8 位哈希）。

第五部分　结果与分析三集合 top1 准确率对照表；混淆矩阵截图；易混类别的原图对比与原因分析；错误样本分类（标注错误 / 定义模糊 / 拍摄质量 / 真实困难）；本组方案的局限性。

第六部分　局限与改进如实说明：哪类样本偏少、哪类背景偏单一、哪组配置未做对比、如果重做会怎么改。**这一部分不扣分，反而加分**——能识别自己方案的短板，是工程素养的体现。

15.4 提交方式与截止

按课程平台要求提交。压缩包体积控制在 500 MB 以内；超限时须先降采样（长边缩至 1280、质量 85），并在 README 中注明。

十六、评分标准

总分 100 分，七个维度。

维度	分值	评分要点
类别设置	12	类别数是否 ≥ 10；类别间形态差异是否可辨；划分依据是否明确并写入 README
样本质量与多样性	18	每类张数是否达标；是否覆盖 6 个拍摄位；是否涵盖不同株、不同开放程度；构图是否规范；是否有明显废片
GPS 完整性	15	抽检 20 张，全部含经纬度得满分；缺失 1—3 张扣 8 分；缺失超过 3 张该项不得分；经纬度不在开封范围内的按伪造处理，本项零分

维度	分值	评分要点
划分正确性	18	比例是否合规；有无数据泄漏（同一植株跨集合）；三子集类别是否齐全；测试集每类是否 ≥ 5 张
结构规范性	12	目录层级是否正确；命名是否合规；压缩包结构是否干净；能否被框架直接加载
训练与可复现	15	训练脚本是否完整可运行；超参是否集中且注释清楚；环境变量设置是否记录；**换机能否复现**；三集合准确率是否齐全；是否固定随机种子
文档与分析	10	`labels.csv` 字段与行数是否匹配；README 六部分是否齐备；混淆矩阵分析是否落到具体图片；局限说明是否真实

加分项（最高 +8 分）

项目	加分	说明
完成一组对照实验并给出解释	+3	如同尺寸 224 vs 320、冻结 vs 全量
类别数超过 12 类且每类仍达标	+2	
额外整理展区采集点分布图（标注经纬度）	+2	
提供可复用的工具脚本（批量重命名、去重、相似度核查等）并在 README 说明用法	+1	

扣分项

情形	处理
使用网络图片或 AI 生成图片	**总分按 0 计**
伪造 GPS 坐标	**总分按 0 计**
组间数据集相似度 ≥ 70%	数据集相关维度按 0 分计（类别、样本质量、GPS、划分四项共 63 分）
组间模型指纹完全相同	训练与可复现维度按 0 分计（15 分）

情形	处理
提交结构无法被框架加载	扣 10 分并退回重做
无法复现（换机跑不通且无合理解释）	扣 8 分
只有准确率数字，无任何图表与分析	文档与分析维度不超过 4 分

附录 A：环境配置速查与排错

A.1 一条命令走完全流程（有 NVIDIA 显卡）

代码

```
python -m venv .venv
.venv\Scripts\activate
pip install torch torchvision --index-url
https://download.pytorch.org/whl/cu121
pip install ultralytics -i https://pypi.tuna.tsinghua.edu.cn/simple
set YOLO_CONFIG_DIR=D:\chrysanthemum\.yolo_config
yolo checks
yolo classify train data=D:\chrysanthemum\datasets\kaifeng_chrysanthemum
model=yolo26n-cls.pt epochs=50 imgsz=224 device=0
```

A.2 环境自检脚本

把下面代码存为 `check_env.py`，运行后一次性输出所有关键信息，**截图放进 README**：

Python 代码

```
import os
import platform
import sys


def main():
    print("=" * 56)
    print("Python 版本 :", sys.version.split()[0])
    print("解释器路径  :", sys.executable)
```

Python 代码

```
    print("操作系统    :", platform.platform())
    print("=" * 56)

    try:
        import torch
        print("PyTorch     :", torch.__version__)
        print("CUDA 可用   :", torch.cuda.is_available())
        print("CUDA 版本   :", torch.version.cuda)
        if torch.cuda.is_available():
            print("显卡        :", torch.cuda.get_device_name(0))
            mem = torch.cuda.get_device_properties(0).total_memory / 1024
** 3
            print(f"显存        : {mem:.1f} GB")
    except ImportError:
        print("PyTorch     : 未安装")

    try:
        import ultralytics
        print("Ultralytics :", ultralytics.__version__)
    except ImportError:
        print("Ultralytics : 未安装")

    for var in ("YOLO_CONFIG_DIR", "CUDA_VISIBLE_DEVICES",
"OMP_NUM_THREADS"):
        print(f"{var:<20}: {os.environ.get(var, '(未设置)')}")
    print("=" * 56)


if __name__ == "__main__":
    main()
```

A.3 报错对照表

报错关键字	含义	处理
use_libuv was requested	Windows 多卡训练	改单卡 device=0，官方文档明确此变量无法解决
WinError 1455	页面文件太小	增加系统虚拟内存，或减小 batch 与 workers
BrokenPipeError	多进程通信中断	workers=0
ModuleNotFoundError:	只装了 torch	补装：pip install

报错关键字	含义	处理
torchvision		torchvision
UnpicklingError	权重文件损坏	重新下载 yolo26n-cls.pt
AssertionError: No images found	目录里没有可读图片	检查扩展名是否被改名（如 .jpeg 写成 .jpg）
KeyError: 'val'	缺 val 目录	补建 val/ 并按类别放图
RuntimeError: DataLoader worker	子进程异常	workers=0 排查，多半是路径含中文

附录 B：经纬度提取与数据集检查脚本

B.1 用 Windows 属性快速查看（无需编程）

右键图片 → 属性 → 详细信息 → 向下滚动到"GPS"区域，查看"GPS 纬度""GPS 经度"。

B.2 批量提取 EXIF 并生成 labels.csv

把下面代码存为 code/build_labels.py：

Python 代码

```
import csv
import os
from PIL import Image
from PIL.ExifTags import GPSTAGS


def dms_to_deg(dms, ref):
    """度分秒转十进制度"""
    d, m, s = [float(x) for x in dms]
    deg = d + m / 60.0 + s / 3600.0
    if ref in ("S", "W"):
        deg = -deg
    return round(deg, 6)
```

Python 代码

```
def read_gps(path):
    """读取单张图片的经纬度与拍摄时间，读不到返回 None"""
    with Image.open(path) as im:
        exif = im.getexif()
        if not exif:
            return None
        gps_info = exif.get_ifd(0x8825)
        if not gps_info:
            return None
        gps = {GPSTAGS.get(k, k): v for k, v in gps_info.items()}
        try:
            lat = dms_to_deg(gps["GPSLatitude"], gps["GPSLatitudeRef"])
            lon = dms_to_deg(gps["GPSLongitude"], gps["GPSLongitudeRef"])
        except (KeyError, ValueError):
            return None
        dt = exif.get(306, "")
        alt = gps.get("GPSAltitude")
        return {
            "latitude": lat,
            "longitude": lon,
            "altitude": round(float(alt), 1) if alt else "",
            "datetime": str(dt).replace(":", "-", 2),
        }


def build_labels(root=".", out_csv="labels.csv"):
    """遍历 train/val/test，生成元数据表"""
    rows = []
    for split in ("train", "val", "test"):
        split_dir = os.path.join(root, split)
        if not os.path.isdir(split_dir):
            continue
        for cat in sorted(os.listdir(split_dir)):
            cat_dir = os.path.join(split_dir, cat)
            if not os.path.isdir(cat_dir):
                continue
            for fn in sorted(os.listdir(cat_dir)):
                if not fn.lower().endswith((".jpg", ".jpeg", ".png")):
                    continue
                info = read_gps(os.path.join(cat_dir, fn))
                if info is None:
                    print(f"[缺少 GPS] {split}/{cat}/{fn}")
                    continue
                rows.append({
                    "filename": fn,
                    "split": split,
                    "category": cat,
```

Python 代码

```
                    "plant_id": "",          # 需手工补填或按序号规则生成
                    "latitude": info["latitude"],
                    "longitude": info["longitude"],
                    "altitude": info["altitude"],
                    "datetime": info["datetime"],
                    "device": "",            # 可手工补填
                })
    with open(out_csv, "w", newline="", encoding="utf-8-sig") as f:
        writer = csv.DictWriter(f, fieldnames=[
            "filename", "split", "category", "plant_id",
            "latitude", "longitude", "altitude", "datetime", "device"])
        writer.writeheader()
        writer.writerows(rows)
    print(f"已写出 {out_csv}，共 {len(rows)} 条记录")


if __name__ == "__main__":
    build_labels(".", "labels.csv")
```

B.3 数据集结构自查

把下面代码存为 `code/check_dataset.py`，训练前必跑一次：

Python 代码

```
import os
from collections import defaultdict


def check(root="."):
    print("=" * 60)
    table = defaultdict(dict)
    for split in ("train", "val", "test"):
        p = os.path.join(root, split)
        if not os.path.isdir(p):
            print(f"[错误] 缺少目录：{split}/")
            continue
        for cat in sorted(os.listdir(p)):
            cdir = os.path.join(p, cat)
            if not os.path.isdir(cdir):
                continue
            n = len([f for f in os.listdir(cdir)
                     if f.lower().endswith((".jpg", ".jpeg", ".png"))])
            table[cat][split] = n
```

Python 代码

```
    cats = sorted(table)
    print(f"类别数：{len(cats)}")
    print(f"{'类别':<28}{'train':>7}{'val':>7}{'test':>7}{'合计':>7}")
    print("-" * 60)
    total = {"train": 0, "val": 0, "test": 0}
    bad = []
    for c in cats:
        row = table[c]
        t, v, s = row.get("train", 0), row.get("val", 0), row.get("test",
0)
        total["train"] += t
        total["val"] += v
        total["test"] += s
        flag = ""
        if s < 5:
            flag = "  ← 测试集不足 5 张"
            bad.append(c)
        print(f"{c:<28}{t:>7}{v:>7}{s:>7}{t + v + s:>7}{flag}")
    print("-" * 60)
    print(f"{'合计
':<28}{total['train']:>7}{total['val']:>7}{total['test']:>7}"
          f"{sum(total.values()):>7}")

    if bad:
        print(f"\n[警告] 以下类别测试集不足 5 张：{bad}")
    if len(cats) < 10:
        print(f"\n[警告] 类别数 {len(cats)} 少于要求的 10 类")


if __name__ == "__main__":
    check(".")
```

B.4 泄漏自查

Python 代码

```
import pandas as pd

df = pd.read_csv("labels.csv")
leak = df.groupby("plant_id")["split"].nunique()   # 同一植株跨越了几个集合
bad = leak[leak > 1]
if len(bad):
    print("发现泄漏，以下植株跨越了多个集合：")
    print(bad)
```

Python 代码

```
else:
    print("未发现泄漏")
```

附录 C：数据集相似度核查脚本

用于核查两组数据集的重合程度，对应 3.2 节的差异化要求。

C.1 原理

用 **dHash（差值哈希）** 把每张图压缩成一个 64 位整数，两图的相似度用汉明距离衡量（不同的位数）。dHash 对压缩、缩放、轻微调色不敏感，但对内容变化敏感——两朵不同的花会得到差异很大的哈希。

为什么不用文件 MD5：MD5 只要有 1 个像素不同就完全不同，改个尺寸就失效，防不住"改一下再交"。

C.2 核查脚本

把下面代码存为 `code/compare_datasets.py`：

Python 代码

```
import os
import sys
from pathlib import Path

import numpy as np
from PIL import Image


def dhash(path, size=8):
    """差值哈希：转灰度 → 缩到 (size+1, size) → 相邻像素比较 → 64 位整数"""
    with Image.open(path) as im:
        im = im.convert("L").resize((size + 1, size), Image.LANCZOS)
        px = np.asarray(im, dtype=np.int16)
    diff = px[:, 1:] > px[:, :-1]
```

Python 代码

```
    bits = 0
    for b in diff.flatten():
        bits = (bits << 1) | int(b)
    return bits


def collect(root):
    """遍历目录下所有图片，返回 [(路径, 哈希)]"""
    exts = (".jpg", ".jpeg", ".png", ".bmp", ".webp")
    items = []
    for p in Path(root).rglob("*"):
        if p.suffix.lower() in exts:
            try:
                items.append((p, dhash(p)))
            except Exception as e:
                print(f"[跳过] {p.name}：{e}")
    return items


def hamming(a, b):
    return bin(a ^ b).count("1")


def similarity(root_a, root_b, threshold=8):
    """相似度 = 重复图片数 / 较小数据集张数"""
    a = collect(root_a)
    b = collect(root_b)
    if not a or not b:
        print("某一侧没有找到图片")
        return None
    dup = 0
    for pa, ha in a:
        for pb, hb in b:
            if hamming(ha, hb) <= threshold:
                dup += 1
                break
    base = min(len(a), len(b))
    return dup, base, dup / base * 100


def main():
    if len(sys.argv) < 3:
        print("用法：python compare_datasets.py <本组数据集> <他组数据集> [阈
值]")
        print("示例：python compare_datasets.py D:\\groupA D:\\groupB 8")
        return
```

Python 代码

```
    root_a, root_b = sys.argv[1], sys.argv[2]
    thr = int(sys.argv[3]) if len(sys.argv) > 3 else 8

    res = similarity(root_a, root_b, thr)
    if res is None:
        return
    dup, base, pct = res
    print("=" * 52)
    print(f"阈值（汉明距离）: {thr}")
    print(f"重复图片数       : {dup}")
    print(f"基准张数         : {base}")
    print(f"相似度           : {pct:.1f}%")
    print("-" * 52)
    if pct < 30:
        print("判定：通过（正常重合）")
    elif pct < 70:
        print("判定：警告（需说明重合原因）")
    else:
        print("判定：不通过（数据集雷同）")
    print("=" * 52)


if __name__ == "__main__":
    main()
```

C.3 判据补充：总相似度不够，还要看"完全重复"比例

总相似度会**稀释**局部抄袭。举例：A 组 450 张，B 组直接复制了其中 200 张，其余 250 张自拍。此时总相似度为 200/450 = 44%，落在"警告"区间——但 200 张直接复制已经是明确的违规。

因此判定要**双指标**：

指标	计算方式	通过	警告	不通过
总相似度	重复数 / 较小集张数	< 30%	30%—70%	≥ 70%
完全重复率	汉明距离 ≤ 2 的张数 / 较小集张数	**< 10%**	10%—20%	**≥ 20%**

汉明距离 ≤ 2 意味着两张图几乎逐像素一致（仅压缩差异），必然是同一张

图的复制。**任一指标落入"不通过"即判不通过。**

修改脚本只需把 `threshold` 与 `strict_threshold` 分开统计：

Python 代码

```
def similarity_detail(root_a, root_b, loose=8, strict=2):
    a, b = collect(root_a), collect(root_b)
    dup = dup_strict = 0
    for pa, ha in a:
        best = min(hamming(ha, hb) for _, hb in b)
        if best <= loose:
            dup += 1
        if best <= strict:
            dup_strict += 1
    base = min(len(a), len(b))
    return {
        "loose_pct": round(dup / base * 100, 1),
        "strict_pct": round(dup_strict / base * 100, 1),
    }
```

C.4 使用说明

1. 只比对 `train/` 目录即可，不必扫全量（速度快 3 倍）。
2. 450 张 × 450 张约需 3—5 秒（纯 Python 循环），可接受。
3. 阈值默认 8。若两组用同一台手机拍同一批花、压缩程度不同，可放宽到 10；若担心漏判，收紧到 6。
4. 建议**组间互查**：甲查乙、乙查丙、丙查甲，结果截图存入 README 第六部分。
5. **脚本已验证**：对 10 张原图与"缩放至 200×200、重新压缩"的副本做比对，汉明距离为 **0**，即 dHash 完全不受压缩与缩放影响；无关图片的相似度为 **0%**。这说明"改一下再交"的做法无法规避检测。

C.5 检测能力与局限

变换方式	能否检出	说明

变换方式	能否检出	说明
重新压缩（JPEG 质量变化）	✓ 能	哈希几乎不变
缩放（如 4000px → 800px）	✓ 能	哈希几乎不变
轻微裁剪（<10% 边缘）	✓ 能	主体内容未变
调亮度 / 对比度	✓ 能	灰度化后差异小
水平翻转	△ 部分	可能超出阈值，需人工复核
加滤镜 / 大幅调色	△ 部分	灰度结构变化大时会漏检
完全不同的一张同类花	不会误判	内容差异大，哈希距离远

局限说明：dHash 只反映**灰度结构的粗粒度布局**。若有人刻意重绘、加边框、拼接，可能规避检测。因此本项核查是**辅助手段**，最终判定仍由教师结合原始 EXIF 拍摄时间、GPS 轨迹、设备型号综合判断。**EXIF 中时间连续、GPS 相邻、设备相同的一组照片，才算真正在同一次外业中采集。**

附录 D：数据规模参考

类别数	每类张数	总张数	适用场景	预期效果
5	20	100	课堂演示	仅能跑通流程
10	30	300	**本任务下限**	YOLO26n 可达 75%—88%
10	45	450	**本任务推荐**	可达 85%—95%
12	45	540	优秀作业	可达 82%—92%
15	40	600	上限	78%—88%，标注风险上升
20	30	600	不推荐	测试集每类仅 3 张，指标不可信

说明：上述精度区间基于 YOLO26 分类模型迁移学习的经验估计，实际结果受类别难度、光照一致性、拍摄多样性影响较大，**不作为评分依据**。评分只看过程的规范性与分析的深度。

附录 E：名词解释

术语	英文	含义
EXIF	Exchangeable Image File Format	图片的可交换图像文件格式，用于存储拍摄参数、时间、GPS 等元数据
GPS	Global Positioning System	全球定位系统，此处指手机定位写入的经纬度
数据泄漏	Data Leakage	训练集与测试集存在信息重叠，导致评估结果虚高
细粒度分类	Fine-grained Classification	类别间差异很小的分类任务，如不同菊花品种
类别不平衡	Class Imbalance	各类样本数量差异大，会导致模型偏向多数类
数据增强	Data Augmentation	通过旋转、裁剪、调色等方式扩充训练样本
迁移学习	Transfer Learning	用在大数据集上预训练的权重作为起点，再在小数据集上微调
微调	Fine-tuning	在预训练权重基础上继续训练，使模型适应新任务
冻结	Freeze	固定部分层的参数不更新，只训练后面的层
早停	Early Stopping	验证指标连续多轮不提升时停止训练，防止过拟合
top1 准确率	Top-1 Accuracy	预测概率最高的类别恰好正确的比率，分类任务的主指标
top5 准确率	Top-5 Accuracy	正确答案出现在概率最高前 5 个类别中的比率
混淆矩阵	Confusion Matrix	按真实类别×预测类别统计的表格，用来看哪两类容易混
感知哈希	Perceptual Hash	把图像内容压缩成短哈希，用于判断两张图是否相似
汉明距离	Hamming Distance	两个等长二进制串中不同位的个数

术语	英文	含义
模型指纹	Model Fingerprint	由关键配置项拼接后取哈希，用于判断两个模型是否相同
随机种子	Random Seed	固定随机数生成器的初始值，使结果可复现
ImageFolder	—	PyTorch 中按目录结构自动加载图像数据集的类
YOLO26	You Only Look Once v26	Ultralytics 的目标检测/分类/分割框架，分类模型以 `-cls` 结尾

本任务书由课程组制定。采集过程请遵守展区管理规定，不攀折花枝、不进入围栏区域；训练过程请遵守学校机房与实验室的使用规定。